Pixel Battle — Combat Animation Overhaul (Design Spec)

Pixel Battle — Combat Animation Overhaul (Design Spec)

> **For agentic workers:** This is a *design spec*. The step-by-step implementation plan is generated separately via `superpowers:writing-plans`. Do not implement directly from this document.

- **Date:** 2026-05-21
- **Status:** Design approved (via brainstorming) — pending implementation plan
- **Origin:** brainstorming session with arlong after reviewing `output/rl_play/episode.mp4`

1. Motivation

The latest RL episode (`pixel_battle/output/rl_play/episode.mp4`) reads as monotonous in combat. Root causes, confirmed by inspecting the renderer:

1. **Rigid single-line arms.** `stick_renderer.py` draws each arm as one straight line (shoulder→hand). With no elbow joint, every "swing" is a stiff stick rotating about the shoulder — motions look small and lifeless.
2. **Only 4 body poses.** The renderer poses by `attack_anim_hint` (`jab` / `kick` / `cooldown` / `special`). But `characters.json` defines 6 characters × 5 skills spanning 8 distinct `vfx` archetypes — so many different skills render as the *same* body pose.
3. **No weapons.** Characters are bare stick figures.
4. The recent "per-skill VFX archetypes" commit improved *effects* (bursts, projectiles) but not the *body motion*.

User feedback, decoded: bigger motions, longer arms, more move variety / swing angles, per-character weapons (e.g. Garen→greatsword, Lux→staff).

2. Goal

A combat-animation overhaul that makes every attack — hit or whiff — read as big, weighty, and visually distinct, with iconic weapons for the LoL champions.

3. Scope & Constraints

In scope

- Procedural 2-segment skeleton (elbow + knee joints) for all 6 characters.
- 8 body poses, one per `vfx` archetype, plus non-attack poses.
- Held weapons for the 4 LoL champions.
- Weapon swing-arc motion smears.
- A `play_multi` curation filter that drops low-action matches.

Hard constraints

- **Renderer-only for the animation work.** No changes to `engine/`, `characters.json`, `env.py`, or the RL observation / action / reward.
- **No RL retraining.** The trained policy checkpoint stays 100% valid; this is "same fight, drawn better."
- `draw_stick_figure(surf, char, color)` keeps its signature → `play.py`'s render loop is untouched *by the animation work*. (`play.py` is touched only for the curation feature — see §10.)

Out of scope (explicitly)

- Changing fight behavior, AI decisions, or gameplay timing.
- New characters or skills.
- Smarter / less-boring AI behavior (corner-camping, jitter). That is a separate effort requiring retraining — see §14.

4. Characters & Weapons

Roster (already in `characters.json`): `brick_phone`, `glass_slab`, `garen`, `lux`, `yasuo`, `ashe`.

Character	Weapon	Silhouette (stick-figure simple)	Grip
garen	Greatsword	long thick tapered filled blade + short crossguard + grip	two-hand
lux	Staff	long thin shaft + glowing orb at tip (accent colour + white core)	one-hand (front); other hand casts
yasuo	Katana	medium thin gently-curved single-edge blade + small round guard	one-hand; two-hand for ult / dash

Character	Weapon	Silhouette (stick-figure simple)	Grip
ashe	Bow	curved bow stave + bowstring; front hand holds, draw hand pulls string in windup	one-hand
brick_phone	— unarmed	—	—
glass_slab	— unarmed	—	—

The skeleton / motion overhaul applies to **all 6**. Weapons apply to the 4 champions only — `brick_phone` and `glass_slab` stay unarmed brawlers (thematically they *are* phones; they headbutt / swipe and already throw or shatter themselves).

5. Architecture

5.1 Skeleton model

Each limb is 2 segments:

- Arm: shoulder → elbow → hand
- Leg: hip → knee → foot

A limb is defined by two angles — the proximal-segment angle and the joint flex angle. Forward kinematics (FK) converts angles → world positions of elbow / hand (and knee / foot). All angles are mirrored by `char.facing`.

Why joints matter: an elbow lets a weapon be cocked fully back during windup and snapped to full extension on strike — that is what makes a motion read as *big*. A knee gives crouch (slam windup), lunge (dash), landing absorption, and a kick chamber.

5.2 File layout

File	Change	Contents
`pixel_battle/rl/poses.py`	**new**	skeleton FK; 8 archetype pose tables; idle / walk / jump / hit / kick poses; angle interpolation
`pixel_battle/rl/weapons.py`	**new**	`_WEAPONS` registry (keyed by `char.id`, mirrors `_STYLES`); per-type weapon drawing; swing-arc smear

File	Change	Contents
`pixel_battle/rl/stick_renderer.py`	**modify**	`draw_stick_figure` orchestrates jointed skeleton + poses + weapons; `_STYLES` extended (longer arms, upper/forearm + thigh/shin splits); `_draw_ghost` upgraded to the jointed pose; VFX helpers & `ProjectileLayer` unchanged
`pixel_battle/rl/play.py`	**modify (curation only)**	`run_one_match` adds an `action_score` to its result dict; the render loop is otherwise untouched
`pixel_battle/rl/play_multi.py`	**modify (curation only)**	accept a match only if it ends in KO **and** `action_score ≥ MIN_ACTION_EVENTS`

`stick_renderer.py` is 523 lines today; folding skeleton FK, 8 × 3-phase pose tables, and weapon drawing into it would push it past ~900 lines. Splitting into `poses.py` + `weapons.py` keeps each unit focused and unit-testable.

5.3 Data flow (per rendered frame)

`play.py` render loop → `draw_stick_figure(surf, char, color)`:

1. `get_style(char.id)` → proportions (extended `_STYLES`).
2. Select pose:
   - `attack_anim_hint == "kick"` → kick pose
   - else if attacking → archetype pose from `char.attack_used_kind.vfx`
   - else → idle / walk / jump / hit by `char.action_state`
3. `poses` computes joint angles for the pose + phase (`attack_phase`, `attack_phase_t`), interpolated with the existing easing functions → FK → world positions.
4. Draw the 2-segment limbs (with hand / foot caps), torso, head.
5. `weapons.get_weapon(char.id)` → if present, draw the weapon at the grip point(s) using the pose's weapon angle; draw the swing-arc smear during the strike phase.
6. Existing fast-movement smear ghosts, upgraded to the jointed pose.

All pose-selection inputs (`attack_used_kind`, `attack_anim_hint`, `attack_phase`, `attack_phase_t`, `action_state`, `facing`, `vel_x`, `on_ground`) already exist on `Character` — verified. No engine field is added.

6. The 8 archetype poses

Each pose is a keyframe set across windup → strike → recover, specifying joint angles for both arms, both legs, torso lean, and weapon angle.

archetype	windup → strike → recover	used by
melee	weapon-arm elbow cocks back → elbow snaps straight into a full-extension horizontal arc → inertial recover with overshoot	all BASIC attacks
slam	both arms raise the weapon overhead, torso leans back, knees crouch → whole body chops straight down, weapon swept past the hip → slow rise	Garen / brick / Yasuo ultimates
spin	both arms extend out wide (elbows straight) → torso rotates a full turn, weapon sweeps a circle, heavy smear	Garen judgment
dash	weapon-arm pulls back, front knee chambers → explosive forward lunge: front knee deep, back leg extended, weapon thrust forward, big lean → recenter	Garen decisive_strike, Yasuo sweeping_blade
bolt	weapon hand sinks / guides back → arm snaps forward, points at the target (projectile spawns at the weapon tip)	most ranged skills
multishot	weapon / bow drawn to one side → sweeps a wide arc, releasing mid-sweep	Ashe volley, glass scatter
aura	weapon planted / one hand down, the other arm raised overhead → slight crouch then rise (“charge up”)	self-buff skills
beam	both arms + weapon pushed fully forward and **held**, wide braced leg stance leaning into the recoil	Lux final_spark, glass force_update

Non-attack poses (also jointed, since legs gained knees): idle (subtle breathing, slight knee bend), walk / run (knee-bent leg swing + elbow counter-swing), jump (knee tuck at apex, legs extend on landing), hit_stagger (arms flail, knees buckle), kick (high knee chamber → shin snaps straight → retract).

Distinctness requirement: each archetype's hand / weapon-tip trajectory must be meaningfully different from the others (slam vertical, beam static-forward, spin full circle, dash forward thrust, melee horizontal arc, bolt short point, multishot wide sweep, aura upward raise). Locked by a test — see §12.

7. Weapons

- `weapons.py` holds `_WEAPONS`, a registry keyed by `char.id`, mirroring the existing renderer-side `_STYLES` dict. Rationale: visual / style data lives renderer-side (as `_STYLES` already does); gameplay data stays in `characters.json`. This keeps the work zero-engine-change.
- A weapon is gripped at the hand. For two-handed weapons, both hands are placed onto the haft by the pose; the bent elbows let both arms reach naturally.
- **Weapon angle is authored per pose-phase**, not auto-derived from the forearm direction. This gives deliberate, controllable swing angles (the slam's blade is vertical overhead in windup, vertical down on strike; melee sweeps a horizontal arc).
- Weapon stroke width scales with the character's `line_width`. Weapons are silhouette-readable, not detailed.
- Draw order: back arm / leg → torso → head → front leg → weapon → front-arm grip, so the weapon layers correctly.

8. Swing-arc smear

During the strike phase of `melee`, `slam`, `spin`, `dash`, and `multishot`, draw 2–3 faded copies of the weapon — or, for unarmed brick / glass, the swinging fist / forearm — along its recent angular sweep. This is a motion-blur arc, character-coloured, using the existing alpha-ghost technique (`_draw_ghost`). It makes every swing — hit *or* whiff — read as powerful, directly addressing the original "whiffs look dull" complaint.

9. Visual-safety guarantees

The renderer enforces these, each locked by a test (§12), so that neither the new poses nor a future implementer can break the frame:

- **Stays in frame.** Every pose's maximum extent (slam overhead, dash lunge, spin full-extension, weapon tip) is bounded. `play.py`'s camera shows a fixed 502-world-px-tall view (`CAM_VIEW_H`) with the floor framed ~82% down; poses must keep the figure + weapon within that view. Weapon length is accounted for.
- **No broken joints.** FK clamps elbow / knee flex to anatomically plausible ranges — no backward hyperextension.
- **Feet planted.** When `on_ground`, the support foot stays at ground Y; crouches lower the hip via knee flex, never by sinking the whole figure into the floor.
- **Weapon doesn't clip.** Two-handed grips keep both hands on the haft; draw order is correct.

- **No snapping.** All transitions interpolate via easing; facing flips are handled smoothly.

10. play_multi curation filter

`play_multi.py` currently keeps a match only if it ends in KO. Extend it to also drop **low-action** matches:

- `run_one_match` (in `play.py`) tallies an `action_score` for the match — the total count of attack-strike events across both fighters, gathered as it processes the match — and returns it in the `result` dict.
- `play_multi.py` accepts a match only if `result["finished_by_ko"]` **and** `result["action_score"] ≥ MIN_ACTION_EVENTS`. A low-action match is dropped and the loop advances to the next seed, exactly like the existing non-KO skip.
- `MIN_ACTION_EVENTS` is a named module constant, tuned by watching output (starting value ≈ 12; adjust during validation).

This is recorder-side only — no engine change, no retrain.

11. Error handling

- Unknown `vfx` archetype → fall back to the `melee` pose.
- `attack_used_kind` is `None` while attacking → fall back to `melee`.
- No weapon registered for `char.id` → render unarmed (skip the weapon draw).
- Joint-angle interpolation `t` is clamped to [0, 1] (the easing functions already do this); FK itself needs no special-casing — it is pure trigonometry.

12. Testing

Existing tests stay green: `test_stick_renderer_pose.py`, `test_skill_vfx.py`, `test_play_richness.py`, `test_play_multi_imports.py`.

New tests (pure functions where possible, no pygame surface needed):

- **FK** — a straight arm (zero flex) → hand at full reach = upper + forearm length; a bent arm → hand nearer; the clamp is enforced.
- **Pose selection** — each of the 8 `vfx` archetypes → a distinct pose; an unknown `vfx` → `melee`; `attack_anim_hint == "kick"` → the kick pose.
- **Distinctness (anti-monotony lock)** — render each archetype's strike frame; assert hand / weapon-tip trajectories differ meaningfully between archetypes.
- **Weapons** — `get_weapon` returns the correct weapon per champion and `None` for brick / glass; the grip point coincides with the hand position.
- **Visual safety** — for every pose × phase: the support foot is at ground Y; elbow / knee flex is within range; the figure + weapon bounding box is within the camera view bounds.

- **Curation** — `run_one_match`'s result includes `action_score`; `play_multi` drops a match below `MIN_ACTION_EVENTS`.

13. Validation

Render a fresh `rl_play` episode and a `rl_play_multi` batch; arlong reviews the mp4(s). Success = combat reads as bigger, weightier, and varied; weapons read clearly; no broken frames.

14. Future / separate work

- **Less-boring AI behavior** (no corner-camping, no jitter) requires retraining with a shaped reward — a separate spec, decoupled from this one.
- Per-topic stylistic variety and extra idle flourishes — later.